

HashDit

Smart Contract Security

ListaDao BpnEarnPool & RenaissNavPool

Security Audit Report

ListaDao

Oct 21 2026 – Oct 26 2026

Disclaimer

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Overview

Audit Period	Oct 21 2026 - Oct 26 2026
Overall Risk [Optional]	**Low**
Project Name	ListaDao
Github	https://github.com/lista-dao/lista-new-contracts/pull/77
Commit	feat/partner-vault-bpn-renaiss

Smart Contract List

No.	Contract Name	Verdict	Details
1	CreditFundBase.sol	See Findings	See Findings
2	FlexEarnPool.sol	See Findings	See Findings
3	InterestDistributor.sol	See Findings	See Findings
4	RenaissNavPool.sol	See Findings	See Findings
5	SurfinAdapter.sol	See Findings	See Findings
6	BpnEarnPool.sol	See Findings	See Findings
7	CreditFundSharePool.sol	See Findings	See Findings

Findings

[L01] Redemptions in BpnEarnPool depend entirely on the BOT with no fallback path	
Contract	BpnEarnPool.sol
Severity	Low

Description	**Description**
	In BpnEarnPool the only function that turns a redemption request from pending into claimable is settleRedeem(), which is restricted to the BOT role. A user first calls requestRedeem(), which escrows their shares, and later calls claimRedeem() to receive cash, but claimRedeem() pays only what settleRedeem() has already marked claimable. There is no cancel path, so a user cannot reverse an escrowed request, and there is no permissionless or admin route to settle it.
	JavaScript
	`function settleRedeem(`
	`uint256[] calldata requestIds,`
	`uint256[] calldata shares`
	`) external onlyRole(BOT) whenNotPaused nonReentrant {`
	`require(requestIds.length == shares.length, "length mismatch");`
	`uint256 total;`
	`for (uint256 i = 0; i < requestIds.length; i++) {`
	`Request storage r = requests[requestIds[i]];`
	`require(r.owner != address(0), "unknown request");`
	`uint256 s = shares[i];`
	`require(s > 0 && s <= r.pending, "invalid shares");`
	`r.pending -= s;`
	`r.claimable += s;`
	`total += s;`
	`emit RedeemSettled(requestIds[i], r.owner, s, r.pending);`
	`}`
	`require(total <= withdrawQuota, "exceeds quota");`
	`withdrawQuota -= total;`
	`totalConfirmedUnclaimed += total;`
	`totalEscrowed -= total;`
	`_burn(address(this), total);`
	`}`
	Impact

	This concentrates the availability of every redemption in a single role. If the BOT goes offline, loses its key, or behaves maliciously, all redemptions freeze. The freeze holds even when the pool already contains enough cash to pay a user. The admin function adminTopUp() only raises withdrawQuota and then runs the batch confirmation step, but BpnEarnPool never opens batches, so that step does nothing and no request becomes claimable. Cash can therefore sit in the pool fully covering a user while that user stays unpaid, because only the BOT can assign it. The escrowed shares are trapped as well, since the guard in emergencyWithdraw() that forbids moving the pool's own token stops the admin from returning them. FlexEarnPool does not share this weakness. There, once funding arrives the batch confirms on its own and the user can claim without the BOT, and if the keeper is unavailable the admin can call adminTopUp() to confirm batches so users still claim. BpnEarnPool removes that safety net. Its funds are recoverable only through emergencyWithdraw(), which pulls the backing cash for off chain redistribution, or through a contract upgrade. Because the BOT is a lower trust automation role rather than the governance multisig, the ability of all users to exit rests on one automation key with no in protocol backstop. This is distinct from the already documented BOT discretion over who is paid first under a shortfall; the issue here is that a redemption cannot complete at all without the BOT, even when the cash to settle it is present.
Recommendation	Provide a fallback settlement path so redemptions cannot be frozen by BOT inaction. One option is to let adminTopUp(), or a dedicated function, assign available withdrawQuota to pending requests in first in first out order using the stored requestedAt value, and to allow this call to run permissionlessly once a timeout has passed.
Status	Acknowledged. settleRedeem is unchanged in the fix PR, no permissionless, admin, or timeout-based fallback settlement path was added, so a BOT that is offline, loses its key, or refuses to act still freezes all BPN redemptions. ListaDao indicates that the BOT being trusted automation and on the existing admin recovery (emergencyWithdraw to redistribute off-chain, or a contract upgrade), as the finding itself notes. Residual risk, there is no in-protocol guarantee that an escrowed request can complete without the BOT. if an on-chain guarantee is wanted, add FIFO settlement from withdrawQuota plus a post-timeout permissionless call.

[L02] Epoch settlement funding couples a liquidity crunch to NAV manipulation in RenaissNavPool	
Contract	RenaissNavPool.sol
Severity	Low
Description	**Description** Closing a redemption epoch in RenaissNavPool happens in one atomic step: settleEpoch pulls requestedShares * publishedNav from the MANAGER, and only then burns the escrowed shares and makes the requests claimable. The same role controls both the NAV it settles at and the cash it must supply, and there is no partial settlement,

so the amount owed and the amount the operator must fund move together under one actor's control. Because the pool holds no on chain reserve and deposits are fully deployable to the off chain partner, all of that cash must be recalled from the partner before an epoch can close.

JavaScript

```
function settleEpoch() external onlyRole(MANAGER) nonReentrant {
   Epoch storage c = epochs[currentEpochId];
   require(c.state == EpochState.Closed, "epoch not closed");
   require(navUpdatedAt > c.closedAt, "nav not published since close");

   uint256 shares = c.requestedShares;
   uint256 settlementNav = publishedNav;
   uint256 payout = (shares * settlementNav) / NAV_PRECISION;
   c.state = EpochState.Settled;
   c.settlementNav = settlementNav;
   c.payout = payout;
   c.feeRate = feeRate;

   if (shares > 0) {
     totalEscrowed -= shares;
     _burn(address(this), shares);
   }
   if (payout > 0) {
     totalOwed += payout;
     IERC20(asset).safeTransferFrom(msg.sender, address(this), payout);
   }

   emit EpochSettled(currentEpochId, settlementNav, shares, payout);
 }
```

Impact
This coupling produces three concrete hazards.

1. Publishing a higher NAV just before settling, whether a genuine gain or an over mark, raises the funding requirement requestedShares * publishedNav; if the increase is not backed by recallable cash, the operator cannot fund the transfer and the epoch cannot settle, freezing the escrowed shares.

2. When the operator is only partially liquid, the only on chain way to close the epoch is to lower the NAV until requestedShares * publishedNav fits the cash available. Since every request in an epoch settles at that single NAV and there is no slippage or minimum payout protection, this silently pays all redeemers in the epoch less, and a malicious operator can settle redeemers at a depressed NAV while the real value stays deployed at the partner.

	3. Third, because redeemers exit into cash at the settlement NAV while remaining holders stay exposed to future NAVs, an operator can mark the NAV down to fund a crunched settlement cheaply and then restore it afterward, permanently underpaying the cohort that exited during the crunch while holders who stayed recover.
Recommendation	Consider including a partial settlement path so a partially liquid operator can pay a pro rata portion of an epoch at the honest NAV instead of being pushed toward a markdown or a freeze. Operationally, only mark the NAV up at settlement when the gain is realized and recallable, and never settle an epoch at a NAV that the recalled liquidity does not actually back. Else, ListaDao admins should always enforce sufficient liquidity during NAV changes for each withdrawal epoch.
Status	Acknowledged, by procedure, ListaDao calls settleEpoch only after the partner's payment for the whole epoch has arrived, and the Safe pays requestedShares × publishedNav in one transfer. The NAV comes from the partner's validated valuation, the MANAGER multisig that publishes it is trusted with it, and it is never lowered to fit the cash on hand. Pricing every request of an epoch at one settlement NAV is intended, so we do not add a partial settlement path; while the payment is pending the epoch stays Closed and its requests cannot be cancelled, which we accept.

[L03] RenaissNavPool deposit has no slippages or deadline	
Contract	RenaissNavPool.sol
Severity	Low
Description	**Description** deposit mints shares using the nav() of the block in which it executes, and currently accepts no minimum shares out and no deadline. The price nav() is not fixed as the MANAGER can move it at any time through publishNav, which has no bound on the size of the move, and it also glides every block while a NAV release is in progress. As a result, the number of shares a depositor receives can differ from what they saw when they built the transaction, with no floor to protect them.

Description

```javascript
 function deposit(uint256 amount, address receiver) external
whenNotPaused whenDepositNotPaused nonReentrant {
    _checkDeposit(receiver, amount);
    uint256 price = nav();
    uint256 shares = (amount * NAV_PRECISION) / price;
    require(shares > 0, "shares is zero");
    // while a rise is being released the price is below publishedNav,
so the shares are
```

```
    // worth more than `amount` at the published value the cap is
measured on
    require(((totalSupply() + shares) * publishedNav) / NAV_PRECISION <=
cap, "exceeds cap");

    _mint(receiver, shares);
    IERC20(asset).safeTransferFrom(msg.sender, adapter, amount);

    emit Deposit(receiver, amount);
    emit Subscribed(msg.sender, receiver, amount, shares, price);
  }
```

Impact

- In ordinary operation a depositor can simply receive fewer shares than expected because the NAV drifted up between quoting and execution, whether from a routine publish or from a release glide.
- In the worst case, a compromised or ordering capable MANAGER can sandwich a deposit: publish a very high NAV just before it so the depositor receives only dust shares, then restore the NAV afterward. The only existing guard, require(shares > 0), blocks just the exact zero case, so a result of 1 share still succeeds and the depositor loses almost the entire amount, which has already been forwarded to the adapter and now benefits the fund.
- Deposits are not routed through the redemption epoch system, so none of the epoch protections apply to them; there is currently nothing shielding a deposit from an adverse NAV move. This is also inconsistent with LockedEarnPool.requestEarlyRedeem, which already carries minPayout and deadline guards for exactly this situation, a payout priced by a mutable, MANAGER controlled rate, added in response to a prior audit finding.

Note that requestRedeemalso reads the mutable nav() and has no deadline, but it must be treated differently, and a request time slippage or deadline guard on it would not be a fail safe. Unlike deposit, requestRedeem does not price the redemption at request time as the redeemer's actual payout is determined later at settleEpoch, where settlementNav = publishedNav under forward pricing, and it is paid in claimRedeem using that settlement NAV rather than the request time nav().

Recommendation	Consider adding a minimum shares out parameter to deposit and revert when the minted shares would fall below it, and add a deadline parameter so a transaction that sits in the mempool cannot execute much later at a stale or adverse price.
Status	Fixed, now adds a minShares parameter to deposit with require(shares >= minShares, "shares below minimum"), giving depositors the requested slippage floor against NAV drift and the publish/release glide. The secondary deadline suggestion was not added, so it is presumed that depositors will be aware of the pending transaction when performing a deposit.

[L04] Pending redemptions count toward the cap inconsistently across the share pools	
Contract	BPNEarnPool.sol FlexEarnPool.sol
Severity	Low

Description	**Description**
	The new share pools apply the deposit cap to pending redemptions in two different ways.
	• BpnEarnPool escrows requested shares without burning them, so they keep counting toward _capUsage, which is totalSupply, until the request is settled. This correctly keeps the cap occupied while the redeemed principal is still held by the fund.
	• FlexEarnPool instead burns the LP at request time, so totalSupply, and therefore cap usage, drops immediately even though the payout is still owed in the batch queue and tracked separately in totalPendingWithdraw.
	JavaScript
	`function requestWithdraw(uint256 amount) external whenNotPaused nonReentrant {`
	`amount = _resolveAll(amount);`
	`require(amount > 0, "amount is zero");`
	`require(balanceOf(msg.sender) >= amount, "insufficient balance");`
	`// min-withdraw floor with dust exit: a sub-min request must drain the balance`
	`_checkMinWithdraw(msg.sender, amount, amount, balanceOf(msg.sender));`
	`// enforce per-address daily submit limit`
	`_consumeDailyLimit(msg.sender, amount);`
	`// burn LP and enqueue`
	`_burn(msg.sender, amount);`
	`uint256 batchId = _enqueueWithdraw(msg.sender, amount, 0);`
	`emit RequestWithdraw(msg.sender, msg.sender, batchId, amount);`
	`}`
	Impact
	Because of this, with a finite flex cap the active LP can refill back up to the cap while older withdrawals are still draining, so the fund's total obligation, totalSupply + totalPendingWithdraw, can briefly exceed the cap by the amount of pending withdrawals that are not yet funded.. The result is an accounting inconsistency between the two pools rather than a fund loss. T

Recommendation	Consider deciding on a single intended meaning for the cap and apply it uniformly. If the cap is meant to bound the fund's total obligation, make FlexEarnPool count pending withdrawals the way BpnEarnPool counts escrowed shares, for example by measuring cap usage as totalSupply plus totalPendingWithdraw rather than letting the burn at request time free capacity early.
Status	Acknowledged, the divergence is intentional and was documented rather than unified. The FlexEarnPool NatSpec is updated to state that the cap bounds outstanding LP supply only, so a flex request burns its LP and frees capacity immediately while its payout is still owed through totalPendingWithdraw, whereas BPN keeps escrowed shares counted until settlement.

[L05] Transferable flex LP makes off-pool interest attribution gameable and widens the daily-limit bypass	
Contract	FlexEarnPool.sol
Severity	Low

Description	**Description**
	The ERC20 migration turned the flex LP into a freely transferable token. Before the change it was a non-transferable accounting balance that could only move through mint and burn, and CreditFundSharePool does not gate transfers by pause or by whitelist. Interest for flex holders is not paid on the LP, it is distributed off-pool through a cumulative Merkle InterestDistributor that is funded from a finite, per-period pot, with each address's share computed off-chain from how much LP it held.
	
	JavaScript
	`function requestWithdraw(uint256 amount) external whenNotPaused nonReentrant {`
	`amount = _resolveAll(amount);`
	`require(amount > 0, "amount is zero");`
	`require(balanceOf(msg.sender) >= amount, "insufficient balance");`
	
	`// min-withdraw floor with dust exit: a sub-min request must drain the balance`
	`_checkMinWithdraw(msg.sender, amount, amount, balanceOf(msg.sender));`
	
	`// enforce per-address daily submit limit`
	`_consumeDailyLimit(msg.sender, amount);`
	
	`// burn LP and enqueue`
	`_burn(msg.sender, amount);`
	`uint256 batchId = _enqueueWithdraw(msg.sender, amount, 0);`
	
	`emit RequestWithdraw(msg.sender, msg.sender, batchId, amount);`
	`}`

	Impact Transferability adds an instant, friction-free way to change an address's balance, which the off-chain interest calculation must now account for. If that calculation is snapshot-based, a user can acquire a large LP balance by transfer just before the snapshot, be credited a full period of interest for a position held only momentarily, and transfer the LP away immediately after. Because the interest pot is finite, every unit captured this way is taken from honest long-term depositors. A transfer that happens in the middle of a period can also be double-credited or orphaned by a careless calculator. None of this can be prevented on-chain, since interest lives off-chain and the LP is not pause, whitelist, or time gated. The only defense is an off-chain calculation that is time-weighted and aware of Transfer events, so a one-period holder earns close to nothing and a mid-period transfer is split by holding duration. This vector did not exist before the migration, when a balance could change only through mint and burn and the burn path carried withdrawal-queue friction. Transferability also makes the already-known per-address daily-withdraw-limit bypass dynamic. requestWithdraw throttles per msg.sender against FLEX_DAILY_LIMIT, which is 200,000e18. A whale can now split a consolidated position across N addresses on demand and have each withdraw up to the limit, draining the fund's buffer by N times 200,000e18 in a single day and crowding out other depositors' exits. Previously the same split had to be arranged at deposit time by choosing receivers.
Recommendation	Consider the following • Make the off-chain interest calculator time-weighted and Transfer-event-aware, so interest tracks the duration each address actually held LP and a mid-period transfer is attributed by holding time without double-counting or orphaning. If interest fairness must be guaranteed on-chain rather than trusted to the calculator, use a heavier mechanism such as an on-chain balance checkpoint the distributor reads. • Reinstate non-transferability, accepting that both trade away the composability the migration enables. For the daily-limit bypass, treat the per-address throttle as a soft rate limit only and rely on the adapter's reserve and buffer management for real liquidity protection, or move the limit to a basis that split addresses cannot evade.
Status	Fixed, the flex LP is now non-transferable by overriding _update to allow only mint and burn (from == address(0) || to == address(0)). This removes the transfer-based interest-snapshot gaming and the dynamic per-address daily-limit bypass.

[L06] All BpnEarnPool deposits revert while any redemption is unfunded	
Contract	BpnEarnPool.sol, SurfinAdapter.sol
Severity	Low
Description	BpnEarnPool.deposit() sends the asset to the adapter and then calls adapter.deployToSurfin(amount), which forwards the full deposit to the partner wallet. deployToSurfin() was designed for manual deploys by the MANAGER. It requires the amount to be at most maxDeployToSurfin(), which first reserves the unfunded part of

the redemption queue (onDemandUnfunded()), so that a manager cannot deploy cash owed to redeemers.

Because every Bpn deposit is forwarded in full, the adapter normally holds almost no idle cash. As soon as one redemption request is unfunded, the only cash on the adapter is the incoming deposit, and it cannot both be deployed in full and partly kept to cover the queue, so the transaction reverts. The deposit size does not matter: every deposit raises freeIdle() by its own amount, so the deployable limit always falls short of the deposit by the unfunded amount.

Example (floorRate = 0, no fee earmark):
the pool holds 1,000,000 of deposits, all forwarded to the partner, and the adapter balance is 0. User A requests a redemption of 100, which is not yet funded. User B then deposits 10,000:

1) after the 10,000 lands on the adapter, freeIdle() = 10,000;
2) onDemandUnfunded() = 100 and hardFloor() = 0;
3) maxDeployToSurfin() = 10,000 − 100 = 9,900;
4) deployToSurfin(10,000) checks 10,000 <= 9,900, which fails, and the transaction reverts.

Whether B deposits 1, 10,000 or 1,000,000, the deposit reverts. Deposits reopen only after the multisig calls settleRecall() to bring at least 100 of cash back to the adapter.

The team's comments assume that phase 1 has no redemptions, but requestRedeem() has no switch, so any holder can request at any time. A full-balance request is also exempt from minWithdraw. A holder can therefore deposit the minimum amount and immediately redeem it in full to block everyone else's deposits. The principal comes back 1:1, so this costs nothing and can be repeated.

In addition, so that the pool can call deployToSurfin(), it must be granted the adapter's full MANAGER role at deployment. The pool thereby also holds setSurfinWallet(), settleRecall(), setFloorRate() and every other manager permission, far more than it needs.

JavaScript

```
// BpnEarnPool — deposit
function deposit(uint256 amount, address receiver) external
whenNotPaused whenDepositNotPaused nonReentrant {
  _checkDeposit(receiver, amount);

  _mint(receiver, amount);
  IERC20(asset).safeTransferFrom(msg.sender, adapter, amount);
  ISurfinAdapter(adapter).deployToSurfin(amount);
  ...
}

// SurfinAdapter — deployToSurfin
function deployToSurfin(uint256 amount) external onlyRole(MANAGER)
whenNotPaused {
```

```
  require(amount > 0, "amount is zero");
  require(amount <= maxDeployToSurfin(), "exceeds deployable");
  ...
}

// SurfinAdapter — maxDeployToSurfin
function maxDeployToSurfin() public view returns (uint256) {
  uint256 free = freeIdle();
  uint256 reserved = hardFloor() + onDemandUnfunded();
  return free > reserved ? free - reserved : 0;
}
```

Impact

- While any redemption is unfunded, every deposit reverts until the multisig and the BOT have recalled and paid out the cash. Once redemptions open in phase 2, ordinary redemptions will pause deposits frequently.
- Any holder can trigger this at no cost and repeatedly, by depositing the minimum amount and immediately redeeming it in full.
- No funds are lost, but the core deposit function becomes repeatedly unavailable, and every recovery needs manual intervention.

Recommendation

Add a dedicated deploy entry point on the adapter for the pool, behind a role that grants only this function. It deploys only the deployable part and leaves the remainder on the adapter to cover the unfunded queue, so deposits no longer revert. BpnEarnPool.deposit then calls deployFromPool(amount) instead of deployToSurfin(amount):

```JavaScript
function deployFromPool(uint256 amount) external onlyRole(POOL)
whenNotPaused {
  uint256 deployable = maxDeployToSurfin();
  uint256 toDeploy = amount < deployable ? amount : deployable;
  if (toDeploy > 0) { deployedToSurfin += toDeploy;
IERC20(asset).safeTransfer(surfinWallet, toDeploy); }
}
```

If not, consider reverting to the Flex/Locked/RenaissNav model (no atomic deployToSurfin in deposit if same-transaction forwarding is not a hard product requirement. Deposits would just mint and transfer to the adapter; the MANAGER/multisig deploys separately, sized to leave the reserve, exactly as the function was designed.

Status

Fixed, now routes deposit through a new SurfinAdapter.deployFromPool, which deploys only min(amount, maxDeployToSurfin()) and leaves the hard-floor and unfunded-queue reserve on the adapter, so a deposit no longer reverts on the

	reservation. It is callable only by the flex-slot pool (msg.sender == flexPool) with no role, which also removes the former full-MANAGER grant on the adapter. Note that deployFromPool emits DeployToSurfin(toDeploy) for the deployed portion only; the remainder it leaves on the adapter has no event, so off-chain monitoring must infer the reserve from Deposit minus DeployToSurfin.

[L07] RenaissNavPool prices subscriptions at the spot price but redemptions forward, letting new subscribers dilute existing holders	
Contract	RenaissNavPool.sol
Severity	**Low**
Description	RenaissNavPool prices redemptions forward: requestRedeem() records only the shares, and settleEpoch() requires a NAV published after the epoch closes and settles at that publishedNav. Subscriptions, however, are priced at the spot price: deposit() computes shares from nav() at execution and mints them immediately. Because of this asymmetry, whenever nav() is below the true NAV, a subscriber buys shares below their true value, and existing holders bear the difference. The MANAGER can publish a NAV in only two ways, and both open such a window: 1. Released publish (releaseDuration > 0): publishNav() sets publishedNav to the new value at once, while nav() moves linearly from the old price to the new one over the release period. During a rise release (up to 7 days), anyone can subscribe below publishedNav, and the window is publicly visible on-chain. 2. Instant publish (releaseDuration = 0, which the NatSpec recommends for rises): the MANAGER is a multisig, so the publish transaction can be read from the Safe Transaction Service while signatures are collected, and it appears in the public mempool before it lands. Anyone can subscribe at the old price ahead of it. The NatSpec only says that subscribers during a release "capture the part not yet released" and calls a stale NAV an operations matter. It does not mention that an instant publish can be front-run, and it does not require subscriptions to be paused around a publish. Example: Bob holds 1,000,000 shares. The partner's assets have grown to 1,100,000, so the true NAV is 1.10, and the MANAGER is about to publish 1.10. 1) Whether Eve front-runs publishNav(1.10e18, 0) or subscribes right after publishNav(1.10e18, 7 days), she pays 100,000 at nav() = 1.00 and receives 100,000 shares. 2) Total supply becomes 1,100,000 shares and true assets 1,100,000 + 100,000 = 1,200,000, so each share is worth about 1.0909. 3) Bob's position falls from 1,100,000 to about 1,090,909, a loss of about 9,091. Eve's position is worth about 109,091, a gain of about 9,091. 4) The contract values the pool at totalSupply × publishedNav = 1,210,000, 10,000 above the true assets. 5) Even if the MANAGER honestly settles the next epoch at the diluted true NAV of about 1.0909, Eve redeems about 109,091 in cash.

For comparison, if Eve subscribes at 1.10 after the NAV has taken effect, she receives about 90,909 shares worth exactly 100,000, and Bob's position stays at 1,100,000.

```javascript
// RenaissNavPool — deposit
uint256 price = nav();
uint256 shares = (amount * NAV_PRECISION) / price;
...
_mint(receiver, shares);

// RenaissNavPool — publishNav
uint256 fromNav = nav();
...
lockedNav = releaseDuration == 0 ? int256(0) : int256(newNav) -
int256(fromNav);
navReleaseStart = uint64(block.timestamp);
navReleaseDuration = uint64(releaseDuration);
publishedNav = newNav;

// RenaissNavPool — nav
function nav() public view returns (uint256) {
  return uint256(int256(publishedNav) - currentLockedNav());
}

// RenaissNavPool — settleEpoch
require(navUpdatedAt > c.closedAt, "nav not published since close");
...
uint256 settlementNav = publishedNav;
uint256 payout = (shares * settlementNav) / NAV_PRECISION;
```

Impact

- On every NAV rise, part of the gain that belongs to existing holders can go to whoever subscribes before the price takes effect, a real transfer of value.
- The MANAGER cannot avoid this through the choice of publish mode: a release opens a public window lasting days, and an instant publish can be front-run.
- The subscriber still bears later NAV moves, so this is not risk-free arbitrage, but the expected gain is roughly the size of the rise. The whitelist only gates deposits, and shares are freely transferable.

Recommendation

Require subscriptions to be paused in publishNav(), so the multisig must close subscriptions before the new NAV is computed and reopen them after publishing, which rules out front-running at the old price.

In addition, price deposit() at the higher of nav() and publishedNav, so that subscriptions during a release are never below the published NAV. During a drop release, subscribers then pay the higher nav(), a trade-off that is conservative for existing holders.

```javascript
// RenaissNavPool — publishNav
require(depositPaused, "pause subscriptions first");

// RenaissNavPool — deposit
uint256 price = nav() > publishedNav ? nav() : publishedNav;
```

If not, consider making subscriptions forward priced like redemptions, so both sides transact at a NAV fixed after a dealing window closes. The clean form is a deposit epoch (open the window, let users commit USDT with no shares minted, close, publish the NAV, then mint at the post-close publishedNav), ideally a single combined epoch that collects both subscriptions and redemptions and settles both at the one post-close NAV. The negative of this is that NAV must be fixed at publish and cannot be corrected after.

Status	Acknowledged, ListaDao indicates that pricing subscriptions at the moving nav() during a release is the intended model and is stated in the contract notice, and the publishNav NatSpec says to publish a rise with releaseDuration 0 unless subscribers during the release are meant to capture it. Pricing deposits at max(nav(), publishedNav), or forward-pricing subscriptions, would affect the design of the product. Additionally, ListaDao stated that operationally, subscriptions are closed with setDepositPaused(true) at the partner's NAV cutoff, before the new NAV is known, and reopened in the same batch that publishes it.

[I01] Unused inherited functionalities within the RenaissNavPool	
Contract	RenaissNavPool.sol
Severity	Informational
Description	**Description** On the RenaissNavPool stack, several functions inherited from the adapter and the base contract are not part of the product's design but remain callable, and each is a potential misconfiguration footgun. **Impact** 1. Renaiss pays no interest, since all return is expressed in the NAV, so fundInterest, fundInterestFrom, and an interest distributor have no role. if a distributor is wired to this stack and funded, it would hand out interest that a NAV pool has no concept of, misallocating funds. 2. Renaiss takes its fee inside the pool through its own feeRate and feeReceiver, so the adapter's accruedFee and claimFee path is redundant for this stack. settleRecall is degenerate here, where its locked cover branch reverts because

	the pool's withdrawal queue is inert, so the call can only recall cash and reset the deployed book value, and that book value is written but never read by any on chain calculation, so an incorrect value only misleads off chain monitoring. 3. adminTopUphas no _unconfirmedObligation guard and transfers cash into the pool and raises its withdrawQuota, but nothing in Renaiss ever consumes withdrawQuota, since settlement is funded directly through settleEpoch, so the injected cash is stranded until emergencyWithdraw retrieves it. 4. In addition, because setAdapter requires withdrawQuota to be 0, an adminTopUp on an otherwise empty Renaiss pool blocks a legitimate adapter rewire until the parked quota is cleared. None of these cause a loss on their own, but collectively they form a surface of trusted role operations that do nothing useful on this pool and can quietly misallocate funds or lock out an administrative action.
Recommendation	Consider the following: 1. For any Renaiss stack, do not wire an interest distributor and do not call fundInterest or fundInterestFrom, since the product distributes no interest, and treat the adapter's accruedFee and claimFee path as unused with Lista's cut kept at the pool level only. 2. Avoid adminTopUp on this pool, since it strands cash in an unused accounting slot and can block setAdapter; if emergency funding is ever required, route it through the pool's own settlement path instead. 3. Consider adding guards so these operations fail fast on a NAV stack, for example rejecting fundInterest when no interest distributor is configured, and document that settleRecall on this stack is limited to recalling cash with its deployed book value being informational only. 4. More broadly, consider trimming or gating the inherited surface that does not apply to the NAV product so trusted operators cannot invoke paths that have no effect or that misallocate funds.
Status	Fixed. now documents the unused surface in the NatSpec, on the Renaiss stack the interest distributor, fundInterest/fundInterestFrom, the adapter fee path, settleRecall, finishWithdraw, and adminTopUp have no role (and a top-up only parks cash and blocks setAdapter).

[I02] Pause does not freeze the NAV release clock	
Contract	RenaissNavPool.sol
Severity	Informational
Description	**Description** In RenaissNavPool, the NAV release glide is driven entirely by block.timestamp. The currentLockedNav() function computes elapsed = block.timestamp - navReleaseStart

	and returns 0 once elapsed >= navReleaseDuration. Because of this, a pause does not freeze the release clock, and the glide keeps advancing in the background while the contract is paused. During a NAV release, nav() differs from publishedNav, which creates a capture/overpay window. During a rise release, nav() < publishedNav, so a subscriber could buy cheap through deposit() and later redeem at the higher publishedNav, capturing value. During a drop release, nav() > publishedNav, so a subscriber would overpay. Since deposit() is gated by whenNotPaused and also by the separate whenDepositNotPaused / depositPaused flag, pausing blocks the only action that can enter this window. If the pause outlasts the remaining release duration, then by the time of unpause elapsed >= duration, so currentLockedNav() = 0, nav() = publishedNav, and the window is gone. **Impact** This makes pausing a partial and incidental mitigation of the capture/overpay window. It is non retroactive, since it only blocks new entries during the freeze and does not claw back shares already captured before the pause. On its own this is minor and benign, but two qualifications matter. 1. The same clock independent behavior applies to the deposit only depositPaused flag. That flag is the cleaner lever because requestRedeem, cancelRedeem, and claimRedeem are not gated by depositPaused, so it closes subscriptions without freezing exits. A global pause instead freezes exits and claims as well. 2. This is not a mitigation against the MANAGER. The publishNav() function is not paused gated and is restricted only by onlyRole(MANAGER). Each call resets navReleaseStart = block.timestamp and re-arms lockedNav = newNav - nav(). The MANAGER can therefore re-publish during or just before unpause to renew or extend the capture window past the pause. The glide clock is MANAGER controllable and pause independent, so pause protects only against passive third party capture, never against the MANAGER. The practical consequence is that pause must not be documented or relied upon as a control over the capture/overpay window, because the window is bounded by real elapsed time regardless of pause state.
Recommendation	Do not treat or document pause as a control over the capture/overpay window. If closing subscriptions during a NAV release is the intended operational response, use the depositPaused flag rather than a global pause so that user exits and claims are not frozen, and state explicitly in the contract documentation that this only blocks passive third party entries and provides no protection against the MANAGER. If stronger protection against the capture/overpay window is desired, address the window at its source instead of through pausing, for example by pricing subscriptions during an active release in a way that does not diverge from publishedNav in favor of the subscriber, or by restricting how and when publishNav() can re-arm a release so the glide clock cannot be renewed to extend the window.

Status	Acknowledged, ListaDao has indicated that the release follows wall-clock time on purpose, so a pause never moves the price curve. The intended design is that re-arming a release on each publish is the documented combine-and-reset behaviour of publishNav. depositPaused only stops third-party subscriptions; MANAGER is trusted with the NAV and neither flag constrains it.

[I03] settleEpoch inherits the live NAV instead of a committed price	
Contract	RenaissNavPool.sol
Severity	Informational
Description	**Description** The RenaissNavPool.settleEpoch() function sets settlementNav = publishedNav, so an epoch is settled at whatever NAV happens to be live at the moment the settlement transaction is mined rather than at a price the MANAGER explicitly commits to. Both publishNav() and settleEpoch() are restricted to the MANAGER role, and if the MANAGER role is held by more than one address, a routine or scheduled publishNav(), or the mandatory post close re-publish required by the forward pricing discipline, can land between the MANAGER's decision to settle at a given price and the settlement actually executing. When that happens, the epoch silently settles at a different NAV than intended.

Description

JavaScript

```
function settleEpoch() external onlyRole(MANAGER) nonReentrant {
   Epoch storage c = epochs[currentEpochId];
   require(c.state == EpochState.Closed, "epoch not closed");
   require(navUpdatedAt > c.closedAt, "nav not published since close");

   uint256 shares = c.requestedShares;
   uint256 settlementNav = publishedNav;
   uint256 payout = (shares * settlementNav) / NAV_PRECISION;
   c.state = EpochState.Settled;
   c.settlementNav = settlementNav;
   c.payout = payout;
   c.feeRate = feeRate;

   if (shares > 0) {
     totalEscrowed -= shares;
     _burn(address(this), shares);
   }
   if (payout > 0) {
     totalOwed += payout;
     IERC20(asset).safeTransferFrom(msg.sender, address(this), payout);
   }

   emit EpochSettled(currentEpochId, settlementNav, shares, payout);
 }
```

	Impact This is an operational footgun rather than an exploit. The settlement price is inherited implicitly, so a last second NAV change becomes the settlement NAV without any explicit signal, and settling at a stale pre loss NAV can occur by accident instead of as a deliberate choice.
Recommendation	Consider adjusting the signature to settleEpoch(uint256 expectedNav) and require expectedNav == publishedNav, so the MANAGER commits to the exact settlement price.
Status	Fixed. now adds require(publishedNav == expectedNav, "nav changed"), so the MANAGER commits to the exact settlement NAV and any publishNav landing between the decision and execution reverts the settlement instead of silently changing the price.

[I04] Zero floor rate leaves BpnEarnPool interest funding with no reserve band	
Contract	RenaissNavPool.sol
Severity	Informational
Description	**Description** On the Flex and Locked stack, the 3% hard floor doubles as an interest reserve. The adapter's interest availability check subtracts the pending withdrawal queue but deliberately not the floor, so interest can always draw at least the floor band even when the queue would otherwise reserve all idle cash. The BpnEarnPool stack runs with a floor rate of 0, so that band does not exist. With no floor, interest availability reduces to free idle minus the pending redemption queue, and whenever the unfunded BPN queue reaches free idle, interest availability is 0 and fundInterest reverts with no fallback. **Impact** Three consequences follow, all specific to the zero floor single pool configuration. 1. First, fundInterest is fully blockable rather than merely reduced, where a large or inflated redemption queue drives available interest to 0, where on the floored stack it would only fall to the floor amount. 2. Second, the known front run grief, in which a user raises the queue with a permissionless requestRedeem just before a prepared fundInterest, can now block the call entirely instead of only shrinking it. 3. Third, fundInterest can draw idle all the way down to the bare queue reservation, leaving the adapter with no buffer for new redemptions or deployments, whereas the floored stack always keeps its reserve.
Recommendation	Fund interest on the BpnEarnPool stack through fundInterestFrom rather than fundInterest. Because fundInterestFrom pulls external cash and does not run the interest availability check, it cannot be blocked or griefed by the redemption queue,

	does not drain the adapter's idle buffer, and does not pay interest out of commingled principal.
Status	Acknowledged. the mitigation is operational and uses an existing lever. Interest on the BPN stack should be funded through fundInterestFrom, which pulls external cash and bypasses the idle-availability check, so it cannot be blocked or front-run-griefed by the redemption queue and does not drain the adapter buffer.

[I05] Cap and publishedNav must be managed together, since a NAV change silently opens or closes deposits	
Contract	RenaissNavPool.sol
Severity	Informational
Description	**Description** The deposit cap is measured in value terms as totalSupply times publishedNav, so the cap usage of the existing supply is recomputed whenever the MANAGER changes publishedNav, even with no new deposit involved. A NAV increase can push the measured pool value above the cap, after which every deposit reverts as over cap until the cap is raised or the NAV falls back. A NAV decrease does the reverse, quietly reopening deposit capacity. **Impact** The effect is confined to the deposit gate and does not corrupt existing holders, redemptions, or settlement, so it is operational rather than a safety flaw, but it couples the cap to publishedNav, publishing a higher NAV without raising the cap in the same operation silently halts deposits, and publishing a lower NAV silently increases available capacity. This coupling is not described anywhere; the NatSpec only states that the cap uses publishedNav.
Recommendation	Consider documenting the coupling in the deploy and operations runbook so that whoever publishes a NAV reviews the cap in the same action
Status	Acknowledged, the coupling is operational and effect is confined to the deposit gate (no impact on holders, redemptions, or settlement). The NatSpec already documents that the cap is measured on publishedNav. ListaDao has indicated that when capacity should change, a setCap goes into the same batch as the publish.

[I06] A dust redemption can settle to zero assets while its shares are still burned	
Contract	RenaissNavPool.sol
Severity	Informational
Description	**Description** In RenaissNavPool a redemption request is checked at request time using the price at request time. Because settlement uses the NAV published after the epoch closes (forward pricing), the request time check does not guarantee a nonzero claim. If

settlementNav is low enough that r.shares * settlementNav < 1e18, then amount floors to 0 and the holder receives nothing even though their shares were burned. In effect a holder is paid 0 whenever their position settles to less than one atomic asset unit.

```javascript
 function requestRedeem(uint256 shares) external whenNotPaused
nonReentrant returns (uint256 requestId) {
    Epoch storage c = epochs[currentEpochId];
    require(c.state == EpochState.Open, "window closed");
    shares = _resolveAll(shares);
    require(shares > 0, "shares is zero");
    uint256 value = (shares * nav()) / NAV_PRECISION;
    require(value > 0, "request below 1 wei");
    _checkMinWithdraw(msg.sender, value, shares, balanceOf(msg.sender));
```

Impact
Given that the shares are transferable, an address can come to hold a dust position of 1 raw share unit. For example, at a NAV of 1.0 (publishedNav = 1e18) the holder calls requestRedeem for the whole balance; value = floor(1 * 1e18 / 1e18) = 1, which passes the > 0 check, and the full balance dust exit satisfies minWithdraw. The epoch is closed and the MANAGER settles at a NAV of 0.9 (settlementNav = 0.9e18). At claim, amount = floor(1 * 0.9e18 / 1e18) = 0, so the 1 share is burned and the holder is paid 0, with the rounded away value swept to the fee receiver through the epoch remainder.

Outside this dust range the same zero outcome needs a near total NAV collapse, where 0 is the economically correct payout, so a normal position below minWithdraw, for example 10 USDT marked down by a NAV decline, still receives a proportional nonzero amount.

Note that LockedEarnPool has the analogous dust case: _earlyRedeemPayout rounds the penalty up, so a tiny early redeem inside the penalty window, for example amount = 1 raw unit at a penaltyRate of 0.1e18, produces a penalty of 1 and a payout of 0.

Recommendation	Consider documenting this behavior so that users always redeem at least minWithdraw worth of shares and do not allow _checkMinWithdraw to fall into a full redemption of dust shares.
Status	Acknowledged, outside a near-total NAV collapse, a 0 payout only occurs for a sub-atomic dust position. Additionally, ListaDao does not prefer rRejecting full exits of dust as that would leave sub-minimum balances that could never be redeemed, and the full-balance exit exists to prevent exactly that.

[I07] BOT role is granted but inert on RenaissNavPool

Contract	RenaissNavPool.sol
Severity	Informational
Description	**Description** The BOT role is defined in CreditFundBase and granted at initialization. RenaissNavPool.initialize() forwards a _bot address into the base initializer, which

	requires it to be non-zero and grants the BOT role. On RenaissNavPool, that role has no functional effect.
	
	The only place the BOT role is consumed anywhere in the inheritance chain is CreditFundBase.claimWithdraw(), which authorizes either the request owner or a BOT to claim a batch-queue withdrawal. RenaissNavPool does not use the template's batch withdrawal queue at all. Its redemptions run entirely through the epoch mechanism (requestRedeem, closeEpoch, settleEpoch, claimRedeem), and the inherited userWithdrawalRequests array is never populated. The contract's own notice documents that claimWithdraw and getUserWithdrawalRequests are inert on this pool. Because userWithdrawalRequests is always empty here, claimWithdraw always reverts since its index check can never pass, so the BOT authorization branch is unreachable.
	
	JavaScript
	`_grantRole(DEFAULT_ADMIN_ROLE, _admin);`
	`_grantRole(MANAGER, _manager);`
	`_grantRole(PAUSER, _pauser);`
	`_grantRole(BOT, _bot);`
	
	`asset = _asset;`
	`adapter = _adapter;`
	`_poolName = _name;`
	`_poolSymbol = _symbol;`
	
	Impact
	Every function RenaissNavPool defines avoids the BOT role entirely. deposit, requestRedeem, and cancelRedeem are gated only by pause modifiers, claimRedeem is permissionless, and publishNav, openEpoch, closeEpoch, settleEpoch, setFeeRate, and setFeeReceiver are all MANAGER-gated. The BOT role is therefore granted at deployment but dead on this pool. For contrast, on BpnEarnPool the BOT role is live because settleRedeem is restricted to it.
	
	There is no security impact and no funds are at risk. This is a clarity and operational-hygiene issue. A deployer must still supply a non-zero _bot address at initialization because the base initializer requires it. Granting a privileged looking role that does nothing can mislead operators and monitoring into believing a BOT keeper is required or active on this pool, and it enlarges the apparent privileged-role surface for no reason.

Recommendation	Either clearly document that the BOT role is intentionally inert on the NAV pool or consider revoking the BOT role for the RenaissNavPool atomically during deployment via a deployment script.
Status	Fixed. now documents in the NatSpec that the BOT role granted at initialize has no use on this pool. The role is still granted because the base initializer requires a non-zero bot address.

[I08] setMinDeposit and setCap are not cross-checked, so minDeposit > cap silently closes all deposits

Contract	CreditFundSharePool.sol
Severity	Informational

Description	**Description**
	In CreditFundSharePool, setMinDeposit() (inherited from CreditFundBase) and setCap() are set independently with no cross-check between them. The deposit() function requires both amount >= minDeposit and _capUsage() + amount <= cap. If the MANAGER sets minDeposit above cap, no deposit amount can satisfy both conditions at the same time, and every deposit silently reverts with either "exceeds cap" or "deposit below minimum".
	
	JavaScript
	`/// @dev lowering the cap below the current usage simply closes deposits`
	`function setCap(uint256 _cap) external onlyRole(MANAGER) {`
	`require(_cap != cap, "same cap");`
	`cap = _cap;`
	`emit SetCap(_cap);`
	`}`
	
	Impact
	The condition is self inflicted, MANAGER only, and fully reversible by lowering minDeposit or raising cap, so there is no fund risk and no user balances are affected. The substance is a configuration consistency gap rather than a vulnerability. The related limit setters setMinWithdraw() and setDailyLimit() already cross-check each other, and the CreditFundBase NatSpec explains that an un-crossed pair can otherwise leave a holder with no legal withdraw amount. setMinDeposit() has no equivalent validation against cap, and setCap() has no validation against minDeposit. This inconsistency can lead an operator to leave deposits closed without an obvious cause. It is not documented in the contracts and was not reported in any prior audit.

Recommendation	Consider adding a cross-check between minDeposit and cap similar to the existing setMinWithdraw() and setDailyLimit() validation
Status	Acknowledged, ListaDao has indicated that a setter check would need a cap-0 exemption so that it does not block closing a pool. It would also miss the case that comes up in normal operation, when the remaining capacity falls below minDeposit, which the frontend handles by showing deposits as unavailable. Unlike minWithdraw/dailyLimit, no holder's exit depends on this pair, so they prefer to keep the setters independent.

[I09] Pools assume a stable unit of account and are unsafe with a volatile underlying asset	
Contract	All
Severity	Informational
Description	**Description** The underlying asset is a generic ERC20 chosen at deployment, there is no price oracle anywhere, and all accounting (deposits, cap, minDeposit, minWithdraw, interest, and

	the Renaiss NAV) is denominated in that asset. The products are built for a stable unit of account such as USDT, and nothing on chain enforces that. **Impact** If a volatile asset were used, several assumptions break. • Fixed asset-unit thresholds (cap, minDeposit, minWithdraw) drift in real value as the price moves and would need continuous re-tuning. • The rounding and dust bands stop being negligible for a high-value, low-decimal asset. If the off-chain strategy is not denominated in the same asset, a price move between deploy and recall opens an FX gap that the asset-denominated books do not model. • For the 1:1 face-value pools (FlexEarnPool, BpnEarnPool, LockedEarnPool) the no-haircut par model becomes under-collateralized if the asset appreciates against a strategy valued in another currency, and LockedEarnPool additionally forces holders to carry the asset's volatility for the whole lock term. • RenaissNavPool is affected most, whereby its NAV is struck in asset terms, so the per-share NAV swings with the asset's own price even when the strategy is flat, and the spot-subscribe versus forward-settle asymmetry grows from small, stable-asset edges into frequent, large value transfers, while the release smoothing only manufactures larger arbitrage windows. For the current USDT deployment this is informational, but it would be a high-impact issue if a volatile asset were ever paired with a strategy not denominated in that same asset.
Recommendation	Consider keeping the underlying restricted to a standard (non fee-on-transfer, non-rebasing) stable unit of account, assert that asset in the deploy script, and document it, since nothing on chain enforces it.
Status	Acknowledged, ListaDao has indicated that the pools are designed for a standard stablecoin held at par, U for BPN and USDT for Renaiss, with, rebasing and fee-on-transfer assets being out of scope.

[I10] RenaissNavPool.requestRedeem omits the explicit balance check used by the other pools	
Contract	RenaissNavPool.sol
Severity	Informational
Description	**Description** FlexEarnPool.requestWithdraw and BpnEarnPool.requestRedeem both guard the requested amount with an explicit check, require(balanceOf(msg.sender) >= amount, "insufficient balance"), before changing any state. RenaissNavPool.requestRedeem does not but instead relies on the later _escrow call, which transfers the shares to the pool through the ERC20 _update and reverts ERC20InsufficientBalance when the request exceeds the caller's balance.

	JavaScript
	``
	/// @dev lowering the cap below the current usage simply closes deposits
	function setCap(uint256 _cap) external onlyRole(MANAGER) {
	require(_cap != cap, "same cap");
	cap = _cap;
	emit SetCap(_cap);
	}
	``
	
	Impact
	The safety invariant still holds: a caller cannot request to redeem more than they hold, because _resolveAll(type(uint256).max) resolves to exactly balanceOf(msg.sender) and any specific over-balance amount is rejected by the escrow transfer, and because a revert rolls back the whole transaction the c.requestedShares and totalEscrowed increments that run just before _escrow are undone, so no inconsistent state remains. T
	
	It is a code-quality and consistency gap, where the Renaiss path surfaces a lower-level revert reason, it validates the balance only after mutating state (weaker checks-effects-interactions ordering), and it diverges from the pattern the other two share pools follow.

Recommendation	Consider adding require(balanceOf(msg.sender) >= shares, "insufficient balance") to RenaissNavPool.requestRedeem, placed after _resolveAll and before the c.requestedShares and totalEscrowed increments
Status	Fixed. Now adds uint256 balance = balanceOf(msg.sender); require(balance >= shares, "insufficient balance"); after _resolveAll and before the escrow increments, matching FlexEarnPool and BpnEarnPool. The balance is now validated up front with a clear revert reason, and the cached value is reused for _checkMinWithdraw.

[I11] requestWithdraw and requestRedeem read balanceOf multiple times in a single call	
Contract	FlexEarnPool.sol & RenaissNavPool.sol
Severity	Informational
Description	**Description** Across the share pools, balanceOf(msg.sender) is read more than once in the request path, which is an avoidable repeated storage load because the value does not change between the reads, since no state is mutated between them. • In FlexEarnPool.requestWithdraw the balance is read up to three times: once inside _resolveAll when amount == type(uint256).max, once in the explicit balance sufficiency check, and once when calling the minimum withdraw check. • In RenaissNavPool.requestRedeem the same redundancy exists but to a smaller degree. The balance is read inside _resolveAll when shares == type(uint256).max and then read again when calling _checkMinWithdraw, so up to two reads in the full balance path. RenaissNavPool has no explicit sufficiency check because it relies on the escrow transfer in _escrow to revert

	on an insufficient balance, which is why it reads up to two times rather than three. But if [I10] is implemented, this gas optimization can be considered BpnEarnPool.requestRedeem already caches the balance into a local variable for the sufficiency check and the minimum withdraw check, which is the pattern to follow, but even it still performs a separate _resolveAll read in the full balance path, so it also reads the balance twice in that path. **Impact** There is no funds or correctness impact. This is a gas optimization and a consistency improvement across the three share pools.
Recommendation	Consider applying the same balance cache to the FlexEarnPool and RenaissNavPool
Status	Acknowledged. No fix implemented, considering this is only a gas optimization, no code logic impact. RenaissNavPool.requestRedeem now caches the balance for the min-withdraw check (a side effect of the I10 fix), removing one read.

[I12] RenaissNavPool pausing mechanism considerations	
Contract	RenaissNavPool.sol
Severity	Informational
Description	**Description** The pause mechanism in RenaissNavPool has inverted polarity, wherein deposit, requestRedeem, cancelRedeem, claimRedeem, and closeEpoch are whenNotPaused, but publishNav, settleEpoch, and openEpoch are ungated. **Impact** As such, a pause disables users while leaving the MANAGER's pricing/settlement live. 1. Pause shields an adverse settlement (close → pause → publish arbitrary NAV → settle, with redeemers unable to cancel, claim, or observe) and cannot be used to stop a bad settlement; 2. Pause + ungated emergencyWithdraw can strand already-settled claims 3. A lone PAUSER can wedge the whole redemption pipeline by pausing during an open epoch. Also the NAV release clock keeps running under pause.
Recommendation	Consider gating publishNav/settleEpoch/openEpoch with whenNotPaused, if not, document the pause permissions within the RenaissNavPool explicitly.
Status	Acknowledged, ListaDao has indicated that a pause holds user flows and closeEpoch, so it cannot shorten the cancel window, and once an epoch is closed its requests cannot be cancelled whether or not the pool is paused, so a pause therefore gives MANAGER no settlement lever it lacks otherwise. publishNav, openEpoch and settleEpoch stay open to MANAGER so a wrong NAV can be corrected and a funded epoch settled while flows are held, and settleEpoch(expectedNav) now binds the payout to the NAV it was computed at. The closeEpoch NatSpec now documents which MANAGER calls stay open during a pause,